

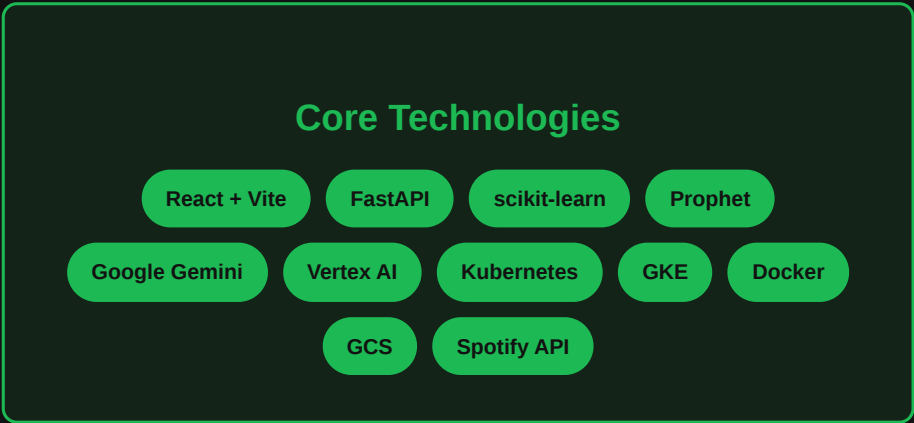
SPOTIFY HIT PREDICTOR

& Trend Forecasting Engine

Architecture & Implementation Document

Full-Stack ML System with GenAI Integration

Kubernetes-Native Deployment on GKE



Project Overview & Vision

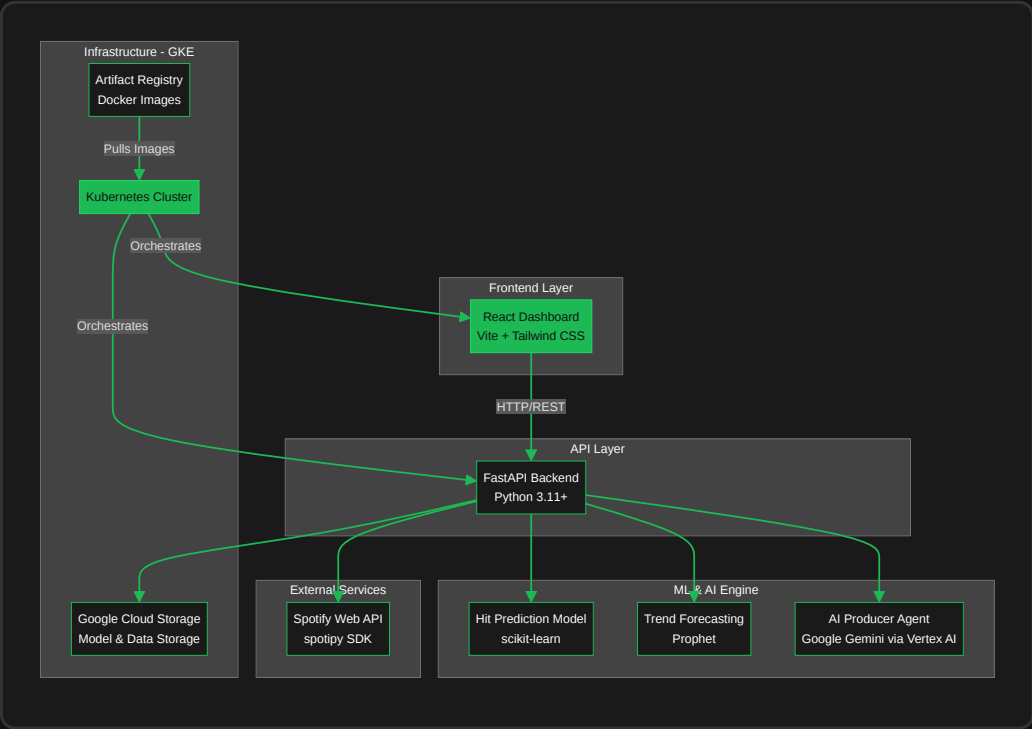
The **Spotify Hit Predictor & Trend Forecasting Engine** is an end-to-end, cloud-native machine learning system designed to predict the commercial success of music tracks and forecast emerging musical trends. The platform integrates traditional ML models with cutting-edge Generative AI capabilities to provide actionable insights for artists, producers, and music industry professionals.

Key Capabilities

- **Hit Prediction:** ML-powered classification model to predict if a track will become a hit based on audio features (danceability, energy, valence, tempo, etc.)
- **Trend Forecasting:** Time-series analysis using Prophet to identify and predict emerging genre trends, seasonal patterns, and market shifts
- **AI Producer Agent:** Gemini-powered conversational interface that provides personalized music production advice, trend insights, and creative recommendations
- **Real-Time Analytics:** Interactive dashboards with visualizations for track analysis, genre trends, and prediction confidence scores

High-Level System Architecture

System Components & Interactions



Technology Stack

Layer	Technology	Purpose
Frontend	React 18+	UI component library
	Vite	Fast build tool and dev server
	Tailwind CSS	Utility-first styling framework
	Chart.js / Recharts	Data visualization
Backend API	FastAPI	High-performance async Python framework
	Pydantic	Data validation and serialization
	Uvicorn	ASGI server
ML & Analytics	scikit-learn	Hit prediction model (Random Forest Classifier)
	Prophet (Meta)	Time-series trend forecasting
	pandas, NumPy	Data manipulation and analysis
GenAI	Google Gemini	LLM for conversational AI producer
	Vertex AI	Google Cloud AI platform integration
External APIs	Spotify Web API	Music data, audio features, playlists
	spotipy	Python SDK for Spotify API
Infrastructure	Google Kubernetes Engine (GKE)	Container orchestration
	Docker	Containerization
	Google Cloud Storage	Model artifacts, datasets, forecasts
	Artifact Registry	Docker image storage
DevOps	Kubernetes (kubectl, Helm)	Deployment management
	Horizontal Pod Autoscaler	Auto-scaling based on load
	CronJob	Scheduled trend data refresh

Project Directory Structure

```

spotify-hit-predictor/ ├── ml-engine/ | ├── collect_data.py # Fetch track data
from Spotify API | ├── collect_trends.py # Collect historical trend data | ├──
train_model.py # Train hit prediction model | ├── train_trends.py # Train Prophet
forecasting model | ├── hit_model.pkl # Serialized scikit-learn model | ├──
trend_forecast.json # Forecast data output | └── requirements.txt | ├── api/ |
| ├── main.py # FastAPI application entry | ├── models.py # Pydantic models/schemas
| | ├── endpoints/ | | ├── predict.py # /predict endpoint | | ├── trends.py #
/trends endpoint | | └── chat.py # /chat endpoint (Gemini) | ├── services/ | |
| ├── ml_service.py # ML model inference | | ├── gemini_service.py # Vertex AI
integration | | └── spotify_service.py # Spotify API client | ├── Dockerfile |
└── requirements.txt | ├── frontend/ | ├── src/ | | ├── components/ # React
components | | ├── pages/ # Dashboard, Predict, Trends, Chat | | ├──
services/api.js # API client | | ├── App.jsx | | └── main.jsx | ├── Dockerfile |
| ├── vite.config.js | └── package.json | ├── k8s/ | ├── deployment-frontend.yaml |
| ├── deployment-api.yaml | ├── service-frontend.yaml | ├── service-api.yaml | ├──
ingress.yaml | ├── hpa.yaml # Horizontal Pod Autoscaler | └── cronjob-trend-
refresh.yaml | └── README.md

```

Development Milestones

Milestone 1: ML & Trend Engine

Objective

Build and train the core machine learning models for hit prediction and trend forecasting.

Components

- `collect_data.py` - Fetch historical track data from Spotify API (audio features, popularity scores, playlist inclusions)
- `collect_trends.py` - Aggregate genre-level trend data over time for time-series analysis
- `train_model.py` - Train Random Forest Classifier on labeled hit/non-hit data using features like danceability, energy, acousticness, valence, loudness, tempo
- `train_trends.py` - Train Prophet model on genre popularity time-series to forecast future trends

Outputs

- `hit_model.pkl` - Serialized scikit-learn model for hit prediction
- `trend_forecast.json` - JSON file containing forecasted trend data for next 6-12 months

Key Metrics

- Model Accuracy: Target 80%+ on test set
- Forecast MAE (Mean Absolute Error): Target < 5% deviation

Technologies

Python

scikit-learn

Prophet

pandas

spotipy

Milestone 2: GenAI Integration

Objective

Integrate Google Gemini via Vertex AI to create an intelligent "AI Producer" conversational agent.

Components

- `gemini_service.py` - Service layer for Vertex AI API integration
- Prompt engineering for music production context (genre advice, mix suggestions, trend insights)
- Context injection: Provide current trend forecasts and model predictions to Gemini for informed responses
- Response parsing and formatting for frontend consumption

Features

- **Conversational Insights:** Users can ask "What genres are trending?" or "How can I make my track more danceable?"
- **Data-Driven Recommendations:** Gemini accesses live trend forecasts and model outputs
- **Creative Guidance:** Suggestions on instrumentation, BPM adjustments, mood tuning

API Endpoint

`POST /chat` - Accepts user query, returns AI-generated response with context from ML models

Technologies

Google Gemini

Vertex AI

Python

FastAPI



Milestone 3: Full-Stack Development

Objective

Build a production-grade FastAPI backend and React dashboard with comprehensive feature set.

Backend API Development (FastAPI)

- `POST /predict` - Accept track audio features, return hit prediction probability and confidence score
- `GET /trends` - Retrieve current and forecasted trend data for multiple genres
- `POST /chat` - Process user queries through Gemini AI Producer agent
- CORS configuration for frontend integration
- Request validation with Pydantic schemas
- Error handling and logging middleware

Frontend Dashboard (React + Vite + Tailwind)

- **Track Prediction Page:** Input form for audio features → hit probability visualization → confidence breakdown
- **Trends Dashboard:** Interactive charts (line graphs, bar charts) showing genre trends over time with Prophet forecasts
- **AI Producer Chat:** Chat interface with message history, streaming responses, context-aware suggestions
- **Analytics Overview:** Summary statistics, popular features analysis, model performance metrics
- Responsive design with Spotify-inspired dark theme (#1DB954 accents)

Integration Testing

- End-to-end API testing with sample data
- Frontend-backend integration validation
- Performance testing (response times, concurrent users)

Technologies

FastAPI

React 18

Vite

Tailwind CSS

Recharts



Milestone 4: Kubernetes Deployment on GKE

Objective

Deploy the entire application stack on Google Kubernetes Engine with auto-scaling, scheduled jobs, and production-grade infrastructure.

Container Images

- Build Docker images for frontend (Nginx serving Vite build) and backend (FastAPI with Uvicorn)
- Push images to Google Artifact Registry
- Implement multi-stage builds for optimized image sizes

Kubernetes Resources

- `deployment-frontend.yaml` - React app deployment (3 replicas)
- `deployment-api.yaml` - FastAPI backend deployment (3 replicas)
- `service-frontend.yaml` - ClusterIP service for frontend
- `service-api.yaml` - ClusterIP service for backend API
- `ingress.yaml` - HTTP(S) load balancer with path-based routing
- `hpa.yaml` - Horizontal Pod Autoscaler (scale 3-10 pods based on CPU/memory)
- `cronjob-trend-refresh.yaml` - Nightly job to retrain trend forecasts and update GCS

Cloud Storage Integration

- Store `hit_model.pkl` and `trend_forecast.json` in Google Cloud Storage buckets
- API pods mount GCS buckets or load models at startup
- CronJob updates forecast data in GCS, API reloads periodically

Deployment Features

- **Auto-scaling:** HPA adjusts pod count based on traffic (3-10 replicas)
- **High Availability:** Multi-replica deployments with readiness/liveness probes
- **Automated Refresh:** CronJob retrains trend model nightly, keeps forecasts fresh
- **Zero-Downtime Updates:** Rolling deployment strategy

Technologies

GKE

Kubernetes

Docker

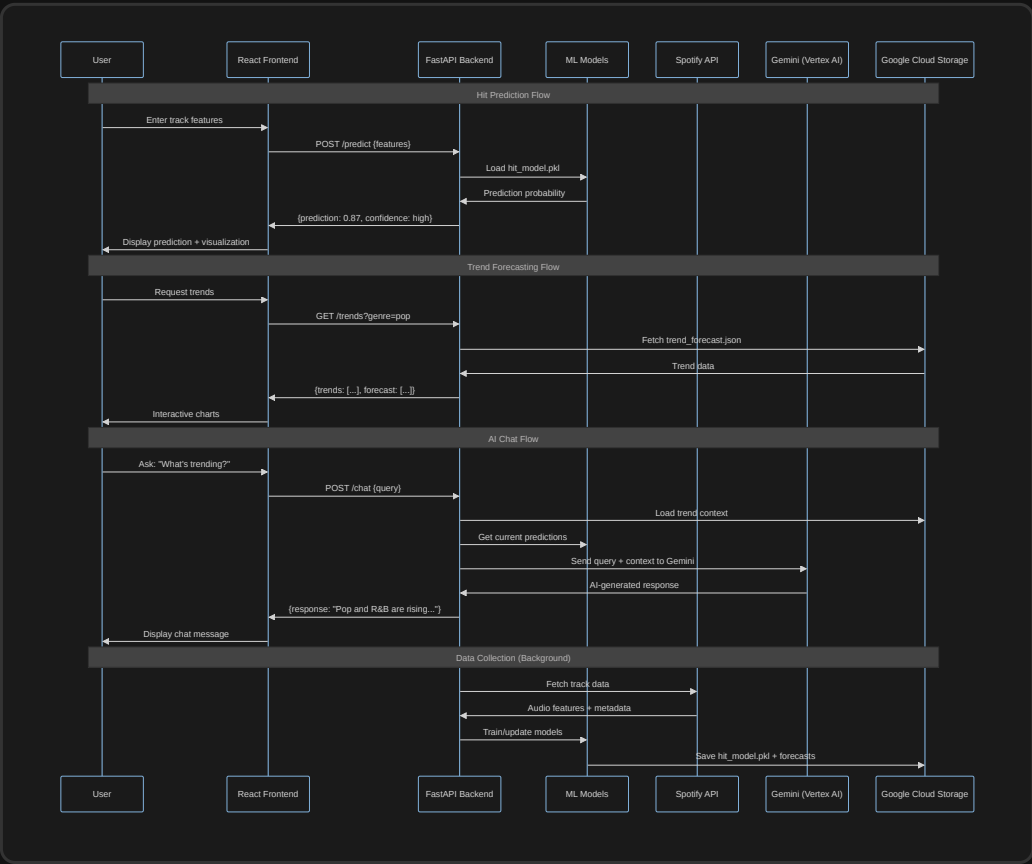
GCS

Artifact Registry

kubectl

Data Flow Architecture

End-to-End Data Flow



API Endpoints Specification

Endpoint	Method	Request Body	Response	Description
<code>/predict</code>	POST	<pre>{ "danceability": 0.75, "energy": 0.82, "valence": 0.65, "tempo": 128, "acousticness": 0.12, "loudness": -5.2}</pre>	<pre>{ "prediction": 0.87, "confidence": "high", "label": "Hit", "features_impact": {...}}</pre>	Pre-beat analysis and prediction
<code>/trends</code>	GET	Query params: <code>?genre=pop&period=6m</code>	<pre>{ "genre": "pop", "historical": [...], "forecast": [...], "trend": "rising", "confidence_interval": {...}}</pre>	Real-time data for trend forecasting
<code>/chat</code>	POST	<pre>{ "query": "What genres are trending?", "context": "production_advice"}</pre>	<pre>{ "response": "Based on current...", "sources": ["trend_model", "gemini"], "suggestions": [...]}</pre>	AI-powered conversation and interactive suggestions
<code>/health</code>	GET	None	<pre>{ "status": "healthy", "models_loaded": true,</pre>	Health check endpoint

Endpoint	Method	Request Body	Response	Description
			<pre>"version": "1.0.0" }</pre>	live prod

Kubernetes Architecture on GKE

GKE Cluster Architecture



Syntax error in text
mermaid version 10.9.5

Kubernetes Manifest Overview

Manifest File	Resource Type	Key Configuration
<code>deployment-frontend.yaml</code>	Deployment	3 replicas - Image: <code>gcr.io/PROJECT/frontend:v1</code> - Port: 80 (Nginx) - Readiness probe: HTTP GET /
<code>deployment-api.yaml</code>	Deployment	3 initial replicas - Image: <code>gcr.io/PROJECT/api:v1</code> - Port: 8000 (FastAPI) - Environment: GCS bucket, Vertex AI credentials - Liveness probe: HTTP GET /health
<code>service-frontend.yaml</code>	Service (ClusterIP)	- Selector: <code>app=frontend</code> - Port: 80 → targetPort: 80
<code>service-api.yaml</code>	Service (ClusterIP)	- Selector: <code>app=api</code> - Port: 8000 → targetPort: 8000
<code>ingress.yaml</code>	Ingress	- Path <code>/</code> → frontend-svc:80 - Path <code>/api/*</code> → api-svc:8000 - TLS/SSL certificate (optional) - Annotations: GCP load balancer class
<code>hpa.yaml</code>	HorizontalPodAutoscaler	- Target: api deployment - Min replicas: 3, Max replicas: 10 - CPU threshold: 70% - Memory threshold: 80%
<code>cronjob-trend-refresh.yaml</code>	CronJob	- Schedule: <code>0 2 * * *</code> (daily 2 AM UTC) - Image: <code>gcr.io/PROJECT/ml-engine:v1</code> - Command: <code>python</code>

Manifest File	Resource Type	Key Configuration
		<code>train_trends.py</code> • Mount GCS bucket for output

Build Order & Execution Plan

Recommended Development Sequence

Follow this systematic build order to ensure dependencies are resolved and integration is seamless.

Phase 1: Foundation & Data Collection

Step 1.1: Spotify API Setup

- Register Spotify Developer account and create application
- Obtain Client ID and Client Secret
- Configure spotipy authentication (Client Credentials Flow)

Step 1.2: Data Collection Scripts

- Implement `collect_data.py`
 - Fetch tracks from popular playlists (Top 50, Viral 50)
 - Extract audio features: danceability, energy, valence, tempo, loudness, acousticness, instrumentalness
 - Label tracks as "Hit" (popularity > 75) or "Non-Hit"
 - Save to CSV: `training_data.csv`
- Implement `collect_trends.py`
 - Aggregate genre popularity over time (monthly snapshots)
 - Save time-series data: `trend_data.csv`

Step 1.3: Verify Data Quality

- Check for missing values, outliers
- Ensure balanced hit/non-hit distribution (aim for 50/50 or apply SMOTE)
- Target: 10,000+ tracks for robust training

Phase 2: ML Model Development

Step 2.1: Hit Prediction Model

- Implement `train_model.py`
- Train-test split (80/20)
- Model: Random Forest Classifier (100-200 estimators)
- Hyperparameter tuning with GridSearchCV
- Feature importance analysis
- Save model: `hit_model.pkl` using joblib/pickle

- **Success Criteria:** Accuracy > 80%, F1-score > 0.75

Step 2.2: Trend Forecasting Model

- Implement `train_trends.py`
- Use Prophet for time-series forecasting
- Train on historical genre popularity data
- Generate 6-month forecasts with confidence intervals
- Save forecasts: `trend_forecast.json`
- **Success Criteria:** MAE < 5% on validation set

Phase 3: Backend API Development

Step 3.1: FastAPI Project Setup

- Initialize FastAPI app in `api/main.py`
- Configure CORS middleware
- Define Pydantic schemas in `models.py`

Step 3.2: Implement Endpoints

- `POST /predict` - Load `hit_model.pkl`, run inference, return prediction
- `GET /trends` - Load `trend_forecast.json`, filter by genre, return data
- `GET /health` - Return API status

Step 3.3: Local Testing

- Run: `uvicorn main:app --reload`
- Test endpoints with curl/Postman
- Validate response schemas

Phase 4: GenAI Integration (Gemini)

Step 4.1: Google Cloud Setup

- Create Google Cloud project
- Enable Vertex AI API
- Set up service account with Vertex AI permissions
- Download credentials JSON

Step 4.2: Gemini Service Implementation

- Implement `gemini_service.py`
- Initialize Vertex AI client with credentials
- Design system prompt for "AI Producer" persona:
 - "You are an expert music producer. Use provided trend data and track analysis to give actionable advice..."
- Context injection: Append trend forecasts and prediction data to user query

Step 4.3: Chat Endpoint

- Implement `POST /chat` in `endpoints/chat.py`
- Load trend context from GCS/local
- Call Gemini API with enriched prompt
- Return formatted response

Step 4.4: Test AI Responses

- Sample queries: "What genres are trending?", "How can I improve track energy?"
- Validate response quality and relevance

Phase 5: Frontend Development

Step 5.1: React Project Setup

- Initialize: `npm create vite@latest frontend -- --template react`
- Install dependencies: Tailwind CSS, Axios, Recharts/Chart.js
- Configure Tailwind with dark theme and Spotify colors

Step 5.2: Build Core Pages

- **Prediction Page:** Form with sliders for audio features → Display prediction result with confidence meter
- **Trends Dashboard:** Fetch `GET /trends` → Render line charts with historical + forecast data
- **AI Chat:** Chat UI with message list → Call `POST /chat` → Display responses
- **Home/Overview:** Stats summary, quick links

Step 5.3: API Integration

- Create `services/api.js` with Axios client

- Configure base URL: `http://localhost:8000` (dev) or Ingress URL (prod)
- Implement error handling and loading states

Step 5.4: Styling & UX

- Dark background (#121212), green accents (#1DB954)
- Responsive design (mobile-first)
- Smooth transitions and animations

Phase 6: Containerization

Step 6.1: Frontend Dockerfile

- Multi-stage build: Node.js for build, Nginx for serving
- Build Vite production bundle
- Copy build to Nginx `/usr/share/nginx/html`

Step 6.2: Backend Dockerfile

- Base: `python:3.11-slim`
- Install dependencies from `requirements.txt`
- Copy ML models (`hit_model.pkl` , `trend_forecast.json`)
- CMD: `uvicorn main:app --host 0.0.0.0 --port 8000`

Step 6.3: Build & Test Locally

- `docker build -t frontend:v1 ./frontend`
- `docker build -t api:v1 ./api`
- Test with `docker-compose` (optional)

Phase 7: Google Cloud Infrastructure Setup

Step 7.1: Cloud Storage Setup

- Create GCS bucket: `gs://spotify-predictor-models`
- Upload `hit_model.pkl` and `trend_forecast.json`
- Set bucket permissions (IAM: Storage Object Viewer for GKE service account)

Step 7.2: Artifact Registry

- Create Docker repository in Artifact Registry
- Authenticate Docker: `gcloud auth configure-docker`
- Tag and push images:
 - `docker tag frontend:v1 gcr.io/PROJECT_ID/frontend:v1`
 - `docker push gcr.io/PROJECT_ID/frontend:v1`
 - Repeat for API image

Step 7.3: GKE Cluster Creation

- Create cluster: `gcloud container clusters create spotify-predictor --num-nodes=3`
- Configure kubectl: `gcloud container clusters get-credentials spotify-predictor`

Phase 8: Kubernetes Deployment

Step 8.1: Create Kubernetes Manifests

- Write all manifests in `k8s/` directory (see Milestone 4 for details)
- Configure environment variables in deployment YAMLs (GCS bucket, Vertex AI project)

Step 8.2: Deploy to GKE

- `kubectl apply -f k8s/deployment-frontend.yaml`
- `kubectl apply -f k8s/deployment-api.yaml`
- `kubectl apply -f k8s/service-frontend.yaml`
- `kubectl apply -f k8s/service-api.yaml`
- `kubectl apply -f k8s/ingress.yaml`
- `kubectl apply -f k8s/hpa.yaml`
- `kubectl apply -f k8s/cronjob-trend-refresh.yaml`

Step 8.3: Verify Deployment

- `kubectl get pods` - Check all pods are Running
- `kubectl get services` - Verify services
- `kubectl get ingress` - Get external IP
- Test endpoints via Ingress IP

Step 8.4: Configure Auto-Scaling

- Verify HPA: `kubectl get hpa`
- Load test API to trigger scaling
- Monitor: `kubectl top pods`

Step 8.5: Schedule CronJob

- Verify CronJob creation: `kubectl get cronjobs`
- Manually trigger test: `kubectl create job --from=cronjob/trend-refresh test-job`
- Check logs: `kubectl logs job/test-job`
- Confirm `trend_forecast.json` updated in GCS

Phase 9: Production Hardening

Monitoring & Logging

- Enable Google Cloud Monitoring (Stackdriver)
- Set up log aggregation from GKE pods
- Create alerts for pod failures, high CPU/memory

Security

- Use Kubernetes Secrets for sensitive data (Spotify API keys, Vertex AI credentials)
- Enable RBAC (Role-Based Access Control)
- Configure network policies for pod-to-pod communication

Performance Optimization

- Optimize Docker images (remove unnecessary packages, use Alpine base images)
- Implement Redis caching for frequently accessed predictions/trends
- Tune HPA thresholds based on real traffic patterns

Project Summary & Next Steps

Key Deliverables

- ✓ **ML Models:** Hit prediction (Random Forest) + Trend forecasting (Prophet)
- ✓ **GenAI Integration:** Gemini-powered AI Producer conversational agent
- ✓ **Full-Stack Application:** React dashboard + FastAPI backend with 3 core endpoints
- ✓ **Cloud Deployment:** Production-ready GKE cluster with auto-scaling and scheduled jobs
- ✓ **Data Pipeline:** Automated trend refresh via Kubernetes CronJob

Architecture Highlights

Component	Technology	Key Feature
Frontend	React + Vite + Tailwind	Spotify-inspired dark theme, interactive charts
API	FastAPI + Uvicorn	Async endpoints, Pydantic validation
ML	scikit-learn + Prophet	80%+ accuracy, 6-month forecasts
GenAI	Google Gemini	Context-aware music production advice
Orchestration	Kubernetes (GKE)	3-10 pod auto-scaling, zero-downtime updates
Storage	Google Cloud Storage	Persistent model artifacts, forecast data
Automation	CronJob	Daily trend model retraining

Success Metrics

Performance KPIs

- Model Accuracy:** Hit prediction > 80% (F1-score > 0.75)
- Forecast Precision:** MAE < 5% on trend predictions
- API Latency:** P95 response time < 200ms
- Uptime:** 99.5% availability (SLA target)
- Scalability:** Handle 100+ concurrent users with HPA

Future Enhancements

Roadmap (Post-MVP)

1. **User Authentication:** OAuth integration for personalized recommendations
2. **Track Upload:** Allow users to upload audio files for analysis (using Spotify's API or librosa)
3. **A/B Testing Framework:** Test model variations in production
4. **Real-Time Streaming:** Live trend updates using WebSockets
5. **Advanced Models:** Experiment with XGBoost, LightGBM, or deep learning (neural networks)
6. **Multi-Region Deployment:** Deploy across multiple GCP regions for global latency reduction
7. **Cost Optimization:** Implement GKE Autopilot, use spot instances for batch jobs
8. **Observability:** Integrate Prometheus + Grafana for custom metrics dashboards

Deployment Checklist

Pre-Production Verification

- ☐ All unit tests passing (backend API, ML models)
- ☐ Integration tests validated (frontend ↔ backend)
- ☐ Docker images built and pushed to Artifact Registry
- ☐ GCS buckets configured with correct IAM permissions
- ☐ Kubernetes manifests reviewed and applied
- ☐ Ingress configured with external IP/domain
- ☐ HPA thresholds tuned based on load testing
- ☐ CronJob tested and verified to update forecasts
- ☐ Monitoring alerts configured (GCP Monitoring)
- ☐ Security: Secrets stored in Kubernetes Secrets, not in code
- ☐ Documentation: README, API docs (Swagger), deployment guide

⚠ Important Notes:

- Spotify API has rate limits (30 requests/second). Implement exponential backoff.
- Gemini API costs scale with tokens. Monitor usage and set budget alerts.
- GKE cluster incurs costs even when idle. Use GKE Autopilot or shut down during development.
- Ensure Vertex AI service account has necessary permissions (Vertex AI User role).

Ready to Build the Future of Music Analytics! 🎵

This architecture provides a scalable, production-grade foundation for an AI-powered music insights platform.

